

National University of Computer and Emerging Sciences
FAST–NUCES

Bachelor of Science in Data Science

Course Code: DS3003 & DS3004

Course Name: Data Warehousing & Business Intelligence

Semester-Fall, 2026

Practical Project

Building and Analysing a Near-Real-Time and Periodic Data Warehouse for a Retail Chain

Total Marks: 100

Weight in grade: 15%

Due date: 13 November 2026, 23:00 hrs.

1. Assessment Task

The student is required to design, implement, and analyse a Data Warehouse (DW) for a retail chain. The chain sells on its website and in its physical stores. The transactional data is therefore of two forms. The first form is the online form. An online customer generates it, and it arrives as a stream. This stream data needs to be loaded into the DW in near real time. The second form is the periodic interval form. It is generated by sales in the physical stores. It is not taken one transaction at a time. It is periodically extracted from Operational Data Sources (ODS) once every hour and then loaded into the DW. Both forms come from one transactional file. You are not given a second file, and you are not given the Master Data (MD). You will prepare the Master Data yourself. A transaction can only be reflected in the DW after completing the whole ETL process. Once all transactional data are reflected in the DW, it must answer the questions asked in the DW Analysis section.

2. Project Overview

The retail chain has a large number of customers. To optimize sales and support promotions, the chain needs to analyze shopping behavior. Online sales must be reflected in the DW soon after they occur. Physical stores' sales must be reflected in the DW periodically (on an hourly basis). An overview is shown in Figure 1.

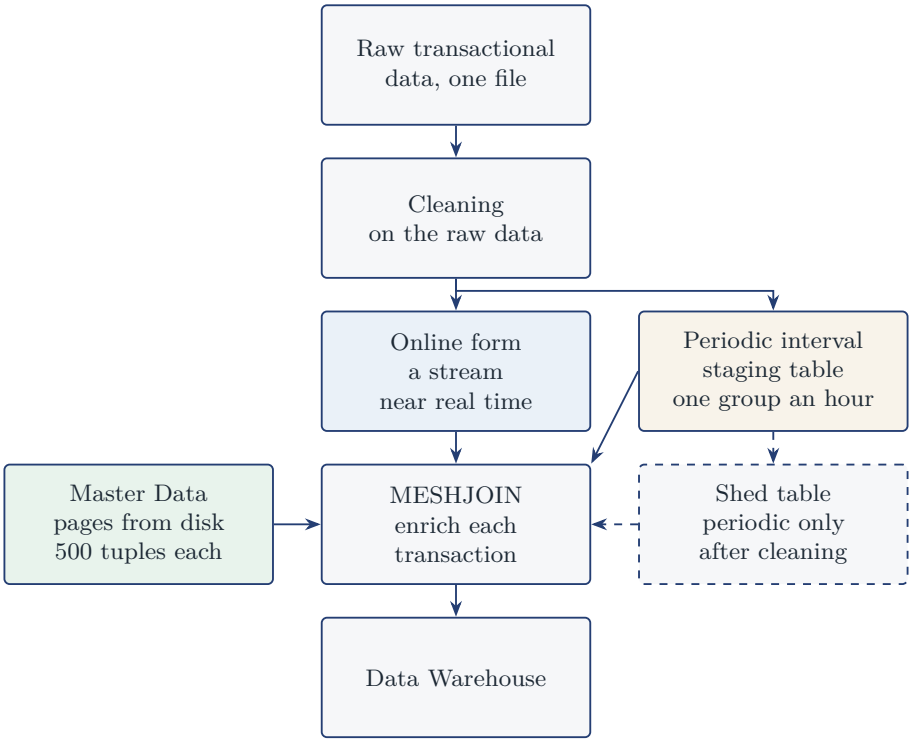


Figure 1: Overview of the DW. Cleaning is first, and it is done on the raw transactional data. Solid arrows are the main path. Dashed arrows move periodic transactions to disk only after cleaning, and only when MESHJOIN is already running at full capacity. These shed transactions are read back when the MESHJOIN is running under capacity.

To build this DW, a near-real-time ETL (Extraction, Transformation, and Loading) is required. The data generated by the customer is incomplete for the DW, so it must be enriched during the transformation phase of ETL. For example, customer demographics or product attributes are taken

from the Master Data (MD), as shown in Figure 2.

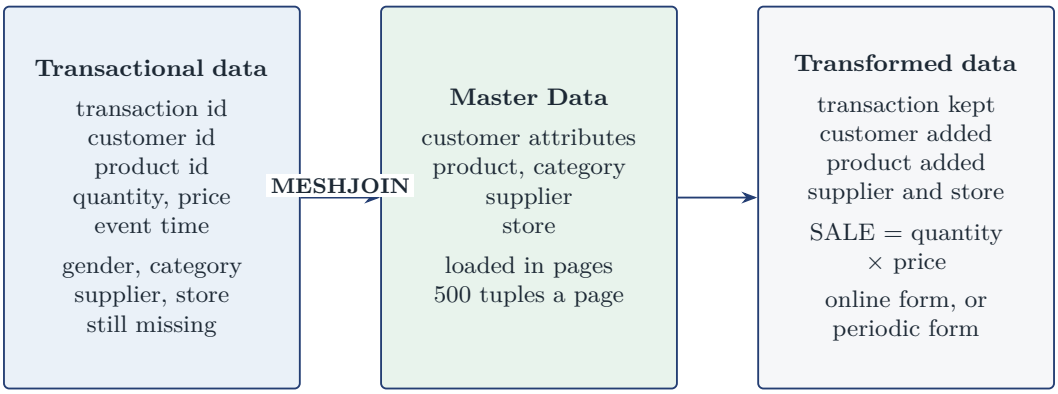


Figure 2: Data enrichment in the transformation phase. It is important to note that SALE does not exist in the transactional data and it does not exist in the Master Data.

A shopping cart is one invoice. One shopping cart can hold many transactions. In this project a transaction means one tuple: one product in one shopping cart, with its quantity, its price, its customer, and its time.

To implement near-real-time enrichment, a Stream-Relation join operator is required in the transformation phase. Several algorithms exist for this join. In this project you will implement MESHJOIN (Mesh Join) in Python.

3. Data Cleaning

You will write a program to clean the transactional file. Do not clean this file by hand before the program reads it. Keep a transaction only when the quantity is greater than zero, the unit price is greater than zero, and the customer ID is present. Drop cancellations. A cancellation is a shopping cart whose `transaction_id` starts with C. Print the number of transactions before these tests and the number after them, and write both counts in the project report.

4. Data Enrichment through MESHJOIN

MESHJOIN was introduced by Polyzotis in 2008. Its objective is the Stream-Relation join in the transformation phase of ETL. You will use it to join the transactional data with the Master Data, and then load the result into the DW.

The stream is called S. S is the sequence of transactions. The Master Data is called R. R stays on disk because it is too large to load as a whole. Do not look up R once for every transaction. That causes many random reads and cannot keep up with the stream. Do not wait until a whole hour is closed before you join. That would stop the stream.

Master Data is loaded in the form of partitions. One partition is one page. Each page has 500 tuples. R is traversed cyclically, in an endless loop.

4.1 MESHJOIN components

Use these names in the report and in the program output. Figure 3 shows the same components.

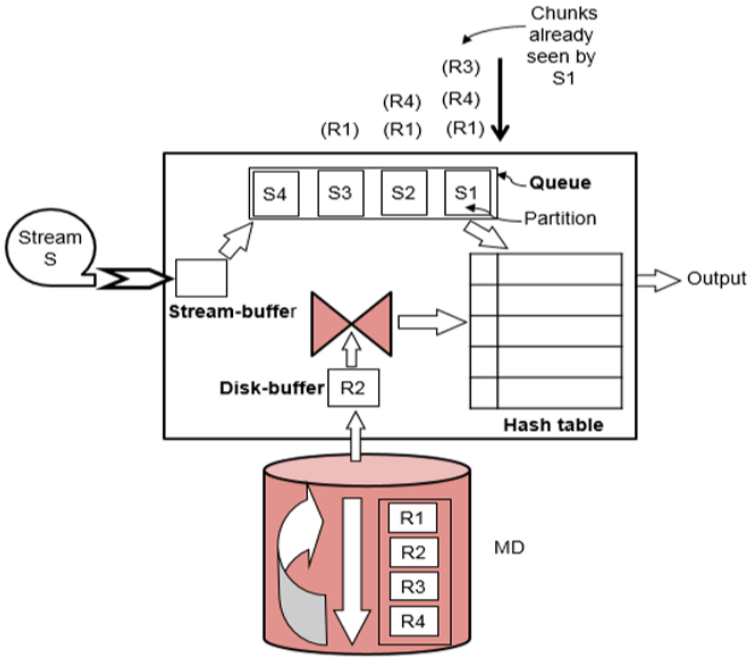


Figure 3: Working of MESHJOIN when partition R2 is in the disk buffer but is not yet processed. The queue keeps the chunks in arrival order. S1 is the oldest chunk.

- **Stream buffer:** An array that holds incoming transactions while MESHJOIN completes one iteration. The stream can arrive faster than MESHJOIN throughput, so a waiting area is needed. The size of the stream buffer is 1000 transactions.
- **Hash table H:** H stores actual sales transactions. It is a multi-map, so many transactions may share one join attribute. When a Master Data tuple is read, look up that join attribute in H.
- **Queue:** The queue keeps a record of the customers' transactions that are in memory, in the order of their arrival. It does not keep a second copy of the transaction. Each node of the queue holds a pointer to one chunk in the hash table. One chunk has $w = 500$ transactions. That chunk is one element of the queue. The oldest chunk is at the end of the queue. The queue has the same number of partitions as the Master Data that is being scanned, so a chunk does not leave until it has been compared with the whole of that Master Data.
- **Disk buffer:** A buffer that holds the one partition of R in memory. The size of the partition is the size of the disk buffer: 500 tuples. The next iteration replaces this partition.
- **Master Data R:** Master Data R is stored on the disk

4.2 MESHJOIN working

The crux of MESHJOIN is as follows. With every iteration, a new chunk of customers' transactions is read from the stream buffer into the hash table, and the Master Data partition in the disk buffer is replaced by the next partition from the disk. Each chunk remains in main memory for one full cycle of the Master Data. The chunks leave main memory in the order in which they entered. Their time in memory overlaps. This is the staggered processing of MESHJOIN.

More precisely, each iteration of MESHOIN implements the following four steps.

1. Read the next partition of the Master Data into the disk buffer. After the last partition, start again at the first partition.
2. Read up to 500 transactions from the stream buffer into H, as one chunk. Put their pointers to

the queue.

3. For each Master Data tuple in the disk buffer, probe its join attribute in H. Every match makes one enriched transaction. The customer pass adds the customer attributes. The product pass adds the product, the supplier, and the store. Keep the quantity, the unit price, the time, and the sales mode (online or periodic flag).
4. Remove a chunk only when its work is finished; that means when it has completed its join with both tables in the master data. If you remove it earlier, you lose matches.

The stream does not wait inside these four steps. A separate thread keeps putting transactions into the stream buffer. The MESHJOIN thread runs the four steps. The periodic thread is described later. These threads do not share one turn inside the cycle. After cleaning, the customer Master Data is a little over 4,000 tuples, about nine partitions, and the product Master Data is a little over 3,000 tuples, about six partitions. The last partition may have fewer than 500 tuples. Count the partitions from the table you loaded, write those counts in the report, and run both passes.

4.3 Priority and Load Shedding

You must need to implement following priority rules for join processing and load shedding.

- Online sales data must be processed on top priority.
- If the MESHJOIN algorithm can't cope with both online and periodic sales arrival rates (this situation can occur in peak hours), the algorithm should shed the periodic sales fully or partially to the disk.
- When the stream arrival rate is slow, and the algorithm has room to process more transactions (usually in idle hours), the shed periodic sales data will be loaded into memory and processed.

4.4 Multi-Threads

The project must implement three independent threads.

- OnlineThread: this thread will run independently to load the online sales data to the MESHJOIN stream buffer.
- MeshJoinThread: this thread implements MESHJOIN and the load shedding modules.
- PeriodicThread: This thread deals with the periodic sales data. It activates once every simulated hour. It extracts the next periodic chunk of sales data from the staging table and loads it into the MESHJOIN stream buffer.

5. Dataset

5.1 Transactional Data

You will use the following link from the UCI Machine Learning Repository to generate both online and periodic sales data.

`https://archive.ics.uci.edu/dataset/352/online+retail`

File `Online Retail.xlsx` given at the above link contains shopping carts from 1 December 2010 through 9 December 2011. Map the attributes given in the file to your project as below.

- `InvoiceNo` to `transaction_id`.

- `StockCode` to `product_id`.
- `Description` to `product_name`. The text is often missing or repeated.
- `Quantity` to `quantity`.
- `InvoiceDate` to `event_time`.
- `UnitPrice` to `unit_price`.
- `CustomerID` to `customer_id`.
- `Country` to `country`.

5.2 Generating Online Sales Data

Once you clean the file and mark each shopping cart as described in the Data Cleaning section, you will write the online transactions to a file named `online_sales_stream.csv`. As mentioned earlier, you will write an independent thread that will read this file and generate a stream of sales. Make sure there is a parameter in your thread to control the speed of the stream. The user can control it from the command line. For example, generate 80 transactions per second in peak time, and 10 transactions per second in non-peak time. Print both numbers when the program starts.

5.3 Generating Periodic Sales Data

This is the sales data generated through Brick-and-Mortar stores. You will load this data into the staging table named `periodic_sales_data`. Later, you will write a thread that will extract this data after every hour and load it to the MESHJOIN stream buffer. One hour in this project is 60 seconds of real time, unless the ReadMe states a different scale and the program prints that scale when it starts. The rule is still one extraction every hour. The 60-second hour is only so that a demonstration does not have to wait a real hour.

Extract the transactions in large groups, not one transaction at a time. Sort the `periodic_sales_data` by `event_time` and divide that sorted list into 48 groups. Extract group 0 at the first hour, group 1 at the next hour, and continue through group 47. About 30 percent of the cleaned file is periodic, so each group has about two thousand transactions. In the report, state how many transactions are in the smallest group and how many are in the largest group.

You may extract the first group as soon as the program starts. Every later group waits until the next hour (60 seconds). The staging table stores a released flag, so the same group is not extracted twice. Each transaction of this data must be flagged as Periodic.

5.4 Generating Master Data

The project has two Master Data tables:- customer and product. MESHJOIN must enrich the sales data from both master data tables. Do not load either Master Data table into a Python dictionary and search that dictionary. The Master Data stays on the disk for the whole run. Each iteration reads one partition of each master data table into memory.

The transactional file names the customer and the product, but it does not describe them. You prepare that description yourself. Take the customer ids and the product ids from the transactional file after cleaning. The public file does not contain gender, age, occupation, category, or supplier. Fill those attributes with the rules below, so that the same id always receives the same values. Do not use a random number.

5.4.1 Rules to differentiate Online Vs Periodic Sales Data

The file does not say which shopping cart was bought on the website and which shopping cart was bought in a store. You decide it with one following fixed rule. The rule uses the invoice number, so every transaction in the same shopping cart receives the same sales_mode. The rule is not applied to each product separately.

(int(MD5(invoice#)[:8], 16) % 10

MD5 is a fixed function. The same invoice number always gives the same result. Do the following.

1. Take `transaction_id` as text, in UTF-8.
2. Compute its MD5.
3. Keep the first 8 characters. These characters are hexadecimal.
4. Convert those 8 characters to a whole number.
5. Divide that number by 10 and keep the remainder.
6. If the remainder is 0, 1, 2, 3, 4, 5, or 6, the shopping cart is online. If the remainder is 7, 8, or 9, the shopping cart is from a physical store.

After applying the above rules you will get 70% of the total as online sales data and about 30% as periodic sales data.

5.4.2 Store rules

Use these five stores, and no others.

- 1, Online, city Web.
- 2, London Bridge, city London.
- 3, Manchester Arndale, city Manchester.
- 4, Birmingham Bullring, city Birmingham.
- 5, Edinburgh Princes, city Edinburgh.

An online shopping cart uses store 1. A store shopping cart uses the same MD5 number as the store rule:

```
store_id = 2 + (int(MD5(transaction_id)[:8], 16) % 4)
```

The remainder with 4 picks one of the four physical stores, and adding 2 gives the store id. The same shopping cart always gets the same store.

5.4.3 Customer and product attributes

Define `stable(text, n)` as the MD5 of the text, the first 8 hexadecimal characters converted to an integer, modulo n. Encode the text as UTF-8. Index 0 is the first value in the list.

- Gender, from `stable(customer_id, 2)`: Male, Female.
- Age group, from `stable(customer_id + "age", 7)`: 0–17, 18–25, 26–35, 36–45, 46–50, 51–55, 55+.

- Occupation, from `stable(customer_id + "occ", 10)`: Student, Engineer, Teacher, Executive, Clerk, Healthcare, Retail, Homemaker, Artist, Other.
- City category, from `stable(customer_id + country, 3)`: A, B, C. Country is the country on that customer's transactions. If more than one country appears, use the country on the first cleaned transaction for that customer.
- Years in the current city, from `stable(customer_id + "stay", 5)`: 0, 1, 2, 3, 4+.
- Marital status, from `stable(customer_id + "mar", 2)`: Single, Married.
- Product name: among the cleaned transactions for that `product_id`, keep the longest Description. If it is empty, use `UNKNOWN`.
- Product category, from `stable(product_id + "cat", 10)`: Home Decor, Kitchen, Stationery, Toys, Seasonal Gifts, Bags, Jewelry, Garden, Party, Other.
- Supplier, from `stable(product_id, 5)`, in this order: SUP01 Northwind Supplies, SUP02 Atlantic Gifts Ltd, SUP03 Midlands Wholesale, SUP04 Celtic Craft Co, SUP05 EuroPack Partners.

The same customer id always receives the same attributes. The same product id always receives the same attributes. Apply the rules to the ids that remain after cleaning. Do not choose the values yourself, and do not use a random number.

Keep the working data as follows.

- Online file `online_sales_stream.csv`: `transaction_id`, `customer_id`, `product_id`, `quantity`, `unit_price`, `event_time`, `sales_mode`.
- `periodic_sales_data` stored in staging table: the same attributes, plus `sales_mode`, `release_hour`, and `released`.
- Customer Master Data, on disk: `customer_id`, `gender`, `age_group`, `occupation`, `city_category`, `stay_in_current_city_years`, `marital_status`.
- Product Master Data, on disk: `product_id`, `product_name`, `product_category`, `supplier_id`, `supplier_name`.
- Store_list: `store_id`, `store_name`, `city`. This list will be stored in memory and the enrichment of its attributes to transactional data will be performed outside MESJOIN.
- `Shed_table`, on disk: the transaction, its flag, and the time it was shed.

6. Project Execution

At startup your project should ask for the host, port, database name, user name, and password. Do not write the password into the code. Also ask for the path of the online file. These are connection details, not settings of the join. Before the threads start, print the stream arrival mode (peak, non-peak), how many seconds make one hour, and the stream-buffer limit.

6.1 Commands

The running program must accept the commands below. You type them while the program is running, and `status` shows that the switch worked.

- `online` — online form only. Do not extract periodic groups.
- `periodic` — periodic groups only. Pause the stream reader.
- `both` — both forms. Take Live transactions first.

- `peak` — fast speed. Shedding periodic transactions to disk is allowed.
- `non-peak` — slow speed. processing shed transactions is allowed.
- `status` — print the mode, the speed, the stream-buffer count, the number of transactions in H, and how many transactions are still on the shed disk.
- `quit` — stop the threads and close the program.

6.2 Data consolidation

This is the last preparation step. It makes one DW tuple from a transaction that has finished both MESHJOIN passes. It is important to note that SALE does not exist in the transactional data and it does not exist in the Master Data. Compute it here:

$$\text{SALE} = \text{QUANTITY} \times \text{PRICE}$$

Store that value on the fact under the name `revenue`. The SQL questions must use this stored revenue. Also store:

- the quantity, copied from the transaction,
- the keys of the five dimensions: customer, product, supplier, store, and time,
- `sales_mode` and `processing_mode`, unchanged,
- `transaction_id`.

One fact tuple is one transaction: one product in one shopping cart, with one customer, one store, one supplier, and one time. Do not combine the transactions of a shopping cart into a coarser tuple. Questions 16 and 19 need the separate transactions. Do not store the business totals as a second fact table. Compute those totals in SQL. If a dimension attribute is missing, the total that needs it cannot be computed.

7. Loading the DW

When the transaction has been prepared, load these into the DW. If a dimension value is not already in its dimension table, insert it into the dimension table first before inserting it into the fact table. The periodic sales data must be loaded periodically using the same interval.

8. Star Schema

The star schema, which you will use in this project, is a data modelling technique that is used to map multidimensional decision support data into a relational database. The star schema yields an easily implemented model for multidimensional data analysis while still preserving the relational structures on which the operational database is built.

The star schema represents aggregated data for specific business activities. Using the schema, one can create multiple aggregated data sources that will represent different aspects of business operations. For example, the aggregation may involve total sales by selected time periods, by products, by stores, and so on. Aggregated totals can be total product sold, total sales values by products, and so on. The basic star schema has three main components: facts, dimensions, attributes, and classification levels. Figure 2 can help you to determine the right components for your star schema.

The fact table is in the centre. Each dimension table joins to the fact table by a key that you

create. The script that creates the DW first drops any old copy of these tables, including the view `STORE_QUARTERLY_SALES`, and then creates them with primary keys and foreign keys. Choose the keys by yourself. Draw the star in the report and explain it.

Customer, product, supplier, and store attributes are already defined in earlier sections. Keep the original ids as well. Build the time tuple from `event_time`, and store the time parts as columns, so that a query can group by them without reading the timestamp again. Store the date, the hour, the day of the month, the month, the quarter, the year, the day of the week, a weekday or weekend flag, the season, and the half-year.

The first half is January through June. The second half is July through December. Spring is March through May, summer is June through August, fall is September through November, and winter is December through February. A weekday is Monday through Friday. The weekend is Saturday or Sunday. The quarter is the calendar quarter. The loaded dates cover 2010 and 2011.

9. Implementation Steps

Write the Python program in the same order as the sections above.

1. Create the working tables and the DW. Drop any old copy, including the view `STORE_QUARTERLY_SALES`, before you create the tables.
2. Read the transactional file. Clean it. Prepare the Master Data. Load the Master Data, write the online file, and load the 48 periodic groups into the staging table.
3. Start the three threads.
4. Load the output data to the DW.
5. Keep accepting the commands until the program is closed.

10. DW Analysis

Once the data has been loaded into the DW, analyse the DW with the following OLAP queries. Write one SQL script. The queries must filter, group, and drill down by time and by product. Question 20 is related to the view.

Where a question says revenue, use the `revenue` stored on the fact. Where it says growth or volatility, compute the percentage change from the previous period. Where it says a year, a quarter, or the past six months, use the dates that are actually loaded, and name those dates in a SQL comment.

Q1. Top revenue-generating products on weekdays and weekends with monthly drill-down. Identifies the top 5 products by revenue, split by weekdays and weekends, with monthly breakdowns for the year loaded in the DW.

Q2. Customer demographics by purchase amount with city category. Analyses total purchase amount by gender and age, detailed by city category.

Q3. Product category sales by occupation. Examines total sales for each product category based on customer occupation.

Q4. Total purchases by gender and age group with quarterly trend. Tracks purchase amount by gender and age across the quarters of the loaded year.

Q5. Top occupations by product category sales. Highlights the top 5 occupations driving sales within each product category.

Q6. City category performance by marital status with monthly breakdown. Assesses purchase amount by city category and marital status over the latest six months present in the DW.

Q7. Average purchase amount by stay duration and gender. Calculates the average purchase amount based on years stayed in the current city and gender.

Q8. Top revenue-generating cities by product category. Ranks city categories by revenue inside each product category. Only three city categories exist, A, B, and C, so the result lists those categories.

Q9. Monthly sales growth by product category. Measures month-over-month sales growth percentage for each product category in the loaded year.

Q10. Weekend vs. weekday sales by age group. Compares total sales by age group for weekends versus weekdays in the loaded year.

Q11. Top revenue-generating products on weekdays and weekends, grouped by month. Finds the top 5 products that generated the highest revenue, separated by weekday and weekend sales, with results grouped by month for the loaded year.

Q12. Quarterly store revenue growth. Calculates the revenue growth rate for each store from one quarter to the next, for the loaded year.

Q13. Detailed supplier sales contribution by store and product name. For each store, shows the sales contribution of each supplier, broken down by product name. Group by store, then supplier, then product name.

Q14. Seasonal analysis of product sales. Presents total sales for each product, drilled down by season: spring, summer, fall, and winter.

Q15. Store-wise and supplier-wise monthly revenue volatility. Calculates the month-to-month revenue volatility for each store and supplier pair. Volatility is the percentage change in revenue from one month to the next.

Q16. Top 5 products purchased together. Identifies the top 5 product pairs that appear together most often in the same shopping cart. A shopping cart is one `transaction_id`. Count distinct shopping carts, not repeated pairs inside one cart. This can inform product bundling.

Q17. Yearly revenue by store, supplier, and product with ROLLUP. Uses `ROLLUP` to aggregate yearly revenue by store, supplier, and product, so the result shows the product, the supplier subtotal, the store subtotal, and the grand total.

Q18. Revenue and volume by product for the two halves of the year. For each product, reports quantity sold and revenue for the first half of the year, the second half, and the full year.

Q19. High revenue spikes. For each product, computes the average daily revenue. Returns the days on which that product's revenue is greater than twice its own daily average. In the report, comment on the spikes you find.

Q20. View STORE_QUARTERLY_SALES. Creates a view named `STORE_QUARTERLY_SALES`. It returns total quarterly sales by store. When you read the view, order the tuples by store name and then by quarter. MySQL does not keep `ORDER BY` inside `CREATE VIEW`, so put the ordering on the `SELECT` that reads the view.

11. Tasks Break-Up

The following is the list of tasks that you need to complete for this project.

1. Identify appropriate dimension tables, the fact table, and their attributes for the sales scenario in Figure 2. Based on that, create a star schema for the DW with appropriate primary and foreign keys.
2. Prepare the transactional data and the Master Data, using the rules described in earlier Sections.
3. Implement MESHJOIN in Python. Load the transactional data (both online and periodic) into the DW after joining it with the Master Data.
4. Apply the analyses, asked in Section 10, on the DW using slicing, dicing, drill-down, and a materialized view.
5. Write a project report. Include the project overview, the DW schema, an explanation of MESHJOIN, any three of its shortcomings, the cleaning counts, the sales_mode rule, the shed counts from peak and non-peak hours, and a reflection on what you learned from the project.
6. Write a ReadMe. It must say how to create the database, prepare the data, start the program, use the commands, and run the queries.

12. Settings

Every student uses the same settings, so the runs can be checked in the same way.

- Master Data partition: 500 tuples, read from the database on each iteration.
- Stream chunk w: 500 transactions taken into H on each iteration.
- Stream buffer limit: 1,000 transactions.
- Peak time: 80 transactions per second. Non-peak time: 10 transactions per second. Both are printed at startup, and both can be changed while the program is running.
- One simulated hour: 60 seconds of real time, printed at startup.
- Periodic groups: 48, after sorting by event time.
- What may be shed: Periodic transactions only when MESHJOIN can't cope the arrival rate. These shed transactions should get processed in non-peak hours.
- Priority of execution: Online transactions first, then periodic transactions and lastly the shed transactions.

13. What to Submit

Submit one zip file. Name it with your full name and student id, for example `MalaikaZainab_23i2605_Project`. Put these folders in the zip.

1. **Create-DW:** the SQL script that creates the star schema. The script must remove any pre-existing schema before creating the tables.
2. **Mesh-Join:** the Python project that implements MESHJOIN, the Master Data, both forms of transactional data, and the load into the DW. The program takes the database credentials, and the path of the online file, from the user at execution time.
3. **Queries-DW:** the SQL script containing the OLAP queries.
4. **Data:** `online_sales_stream.csv`, `customer_master_data.csv`, `product_master_data.csv`, and a short note with the source, the rules, the cleaning counts, and the date you downloaded the file.
5. **Project-Report:** the document described in the tasks.
6. **ReadMe:** the steps required to operate the project. If this file is missing, five marks are

deducted.

14. When and Where to Submit

Submit the zip file on Google Classroom by 13 November 2026, 23:00. A submission made up to 24 hours after that time loses 15 percent of the marks. No submission will be accepted after that.

NOTE: Each student must complete the project individually. All source files and report material must be unique and prepared independently. Submissions will be checked using Turnitin, and any copied or identical work will receive zero.

15. Marking Guidelines

Component	Description	Marks
Create-DW	SQL script that creates the star schema. The script drops any existing tables, creates the dimensions and the fact, and applies the primary and foreign keys.	15
Mesh-Join	Python project that implements MESHJOIN. It extracts the transactional data, transforms it with the Master Data, and loads it into the DW. The user types the password at startup. Both the online form and the periodic interval form are required.	30
Queries-DW	SQL script containing the OLAP queries in the DW Analysis section.	30
Project-Report	Overview, DW schema, MESHJOIN and three shortcomings, cleaning counts, sales_mode rule, shed counts, and what you learned.	20
ReadMe	Step-by-step instructions to operate the project. Five marks are deducted if this file is missing.	5

Table 1: Marks. The bonus below has no marks.

16. Bonus

This section is not part of the 100 marks. Do it only after the required work is finished. That means both forms of transactional data, the move of periodic transactions to disk, and all twenty questions.

The bonus changes only the moment at which a Live result enters the DW. Write a Live join result first into a cache table in the database, rather than straight into the fact table. During a slow period, copy those cached transactions into the DW and delete the transactions you copied. Do not cache periodic results. Periodic results still load on the hour, after they have been joined. If you skip the bonus, insert Live results directly into the fact table. Say in the ReadMe which of the two loads you used.

Dataset citation. Put this in the ReadMe and in the report, with the date you downloaded the file. Chen, D., Sain, S. L., and Guo, K. Online Retail. UCI Machine Learning Repository. The address is the link in the Transactional Data section.

«««« ALL THE BEST»»»»